Container Packing Optimizer

Technical Reference & User Manual

Version 1.0 · March 2026

Internal software documentation

Constraint-based 3-D container loading engine

Table of Contents

1 Executive Summary **3**

2 System & Library Requirements **3**

2.1 Hardware & Operating System 3

2.2 Python & Library Dependencies 3

3 Input File Specification **4**

3.1 File Format 4

3.2 Pallet Rows — Required & Optional Columns 4

3.3 NP (Loose Box) Rows 5

3.4 Dimension String Format 5

4 System Architecture **6**

4.1 File Structure 6

4.2 Module Map 6

5 Full Pipeline — Step-by-Step Data Flow **7**

6 Core Algorithms in Detail **8**

6.1 Excel Parsing 8

6.2 Block Type Classification 8

6.3 Constraint Programming Model 9

6.4 NP Loose-Box Assignment 11

6.5 Fill Recommendation Engine 11

7 Configuration Reference **12**

8 Output Files **13**

8.1 report.xlsx 13

8.2 JSON & log files 13

9 Known Assumptions & Limitations **14**

CHAPTER 1

Executive Summary

The **Container Packing Optimizer** is a desktop application that takes a merchandise order in Excel format and computes how to load the pallets and loose boxes into the fewest possible 40-foot high-cube shipping containers, maximising space utilisation. It then recommends which additional product lines to reorder to fill any remaining gaps.

What the software does

- Reads a standard order export from Excel (Edelman format or compatible layout).
- Groups individual pallets of the same footprint and height into *row-blocks* — the physical unit that occupies one full row across the container width.
- Solves a combinatorial optimisation problem (constraint programming) to decide which row-blocks go into each container and in what sequence, maximising pallet count and then length utilisation.
- Fills remaining headroom above each pallet row and the tail zone after the last row with loose (NP) boxes, greedily using all six axis-aligned box orientations.
- Recommends extra units to order from the same product range to fill leftover space.
- Produces a formatted Excel report, JSON data files, a text summary, and (optionally) 3-D matplotlib visualisations.

What the software does not do

- Does not perform true item-level 3-D bin packing — pallets are treated as full-width rows.
- Does not optimise across containers simultaneously; each container is solved sequentially.
- Does not handle non-rectangular or non-stackable items.
- Does not integrate with any ERP or WMS system; all I/O is via Excel files.

CHAPTER 2

System & Library Requirements

2.1 Hardware & Operating System

Component	Minimum	Recommended
CPU	Any modern x86-64	4+ cores (solver is single-threaded but Python is not)
RAM	4 GB	8 GB (large orders with 500+ pallets may need more)
Disk	2 GB free	4 GB (Python environment + OR-Tools binaries)
OS	Windows 10 64-bit, macOS 12+	Windows 11, macOS 14+
Display	1280 × 720	1920 × 1080 (GUI designed for 900 × 680 minimum)

2.2 Python & Library Dependencies

Python **3.10 or 3.11** is required. Python 3.12 is not recommended — some OR-Tools wheel variants may not yet support it.

Library	Version (tested)	Role
`cpmpy`	≥ 0.9.23	Constraint programming modelling layer
`ortools`	≥ 9.8	Google OR-Tools CP-SAT backend solver
`pandas`	≥ 2.0	Excel reading and DataFrame manipulation
`openpyxl`	≥ 3.1	Excel report writing (read_excel + Workbook)
`matplotlib`	≥ 3.8	Optional 3-D container visualisations
`tkinter`	built-in	Desktop GUI (bundled with Python on all platforms)

Install all dependencies in one command from the project root:

```
pip install -r requirements.txt
```

> **Important:** Do *not* install `gurobipy` in the same environment. CPMpy silently attempts to import it at startup and will produce errors if it is installed without a valid Gurobi licence.

For packaging into a standalone executable, `pyinstaller ≥ 6.0` is also required but only on the build machine — it is not shipped with the final application.

CHAPTER 3

Input File Specification

3.1 File Format

The input must be a Microsoft Excel file (`.xlsx` or `.xls`). The sheet can be at any index (default: first sheet). The tool automatically scans for the header row — it looks for any row containing at least two of the following keywords: *Barcode, Pallet size, Pallet and packing size, Productname, Total order full pallets, Total number of pallets, Total pallet in container*. Rows above the detected header are ignored.

A single sheet must contain both pallet rows and NP (loose box) rows. The two row types are distinguished by the value in the **Pallet size** column: rows containing the string `NP` (case-insensitive) are treated as loose-box rows; all other non-empty rows are treated as pallet rows.

3.2 Pallet Rows — Required & Optional Columns

Column group	Accepted column names (prefix-matched, case-insensitive)	Required?	Notes

Dimension string	`Pallet and packing size`
	`Pallet size`
	`size`
		Yes	Must contain dimension string (see §3.4). "Pallet and packing size" takes priority.

Quantity	`External Packaging Quantity`
	`Total order full pallets`
	`Total number of pallets`
	`number of pallets`
	`full pallets`
		Yes	Integer number of full pallets of this type. Rows with zero or missing quantity are skipped.

Product name	`Productname`
	`product name`
	`product`
		No	Used as human-readable label in reports and legends.

Item	`Item`	No	Appended to label if present.

Barcode / EAN	`Barcode`
	`bar code`
	`ean`
		No	Stored in metadata; shown in legend.

Weight per pallet	`External Net weight`
	`Net weight`
	`Weight`
		No	Used for container weight constraint. Defaults to 0 if absent — weight limit will not be binding.

FOB unit price	`Item price FOB`
	`price fob`
	`fob price`
	`unit price`
		No	Used for `max_value` recommendation objective. Ignored otherwise.

3.3 NP (Loose Box) Rows

Any row where the **Pallet size** column equals exactly `NP` is treated as a loose box row. The dimension for NP rows is read from the **Pallet and packing size** column (not the Pallet size column, which just holds "NP"). Quantity for NP rows is resolved in this order:

1. `External Packaging Quantity`

2. `Total number of pallets` (used as box count in NP rows)

Weight per box is read from the same weight columns as pallet rows. Product name, item, and barcode are resolved identically to pallet rows.

3.4 Dimension String Format

All dimensions are parsed from strings like:

```
1.15x1.15x1.01          (metres, dot separator)
1,15x1,15x1,01          (metres, comma separator — European format)
1.15×1.15×1.01          (metres, unicode × separator)
0.35x0.55x0.26          (NP box in metres)
```

The parser multiplies by 100 and rounds to the nearest centimetre. The trailing `cm` suffix is stripped if present. The order of dimensions is **Length × Width × Height**.

> **Example valid row (pallet):** Pallet and packing size = `1.15x1.15x1.01` → parsed as L=115 cm, W=115 cm, H=101 cm.
> **Example NP box row:** Pallet size = `NP`, Pallet and packing size = `0.35x0.55x0.26` → parsed as L=35 cm, W=55 cm, H=26 cm.

CHAPTER 4

System Architecture

4.1 File Structure

```
Container_packing_optimizer/
│
├── app/
│   ├── main.py                    ← Pipeline orchestrator (steps 1–10)
│   ├── gui.py                     ← Tkinter desktop GUI
│   ├── config.py                  ← Configuration loader (JSON + defaults)
│   │
│   ├── models/
│   │   └── A_1D_multi_container_placement_chatGPT.py   ← CP model
│   │
│   └── utils/
│       ├── parse_xlsx.py          ← Excel input parser
│       ├── oneDbuildblocks.py     ← Pallet → row-block classification
│       ├── recommend.py           ← Fill recommendation engine
│       ├── export_excel.py        ← Excel report writer
│       └── visualize_row_blocks.py ← matplotlib 3-D plots
│
├── optimizer_config.json          ← User-editable container config
├── requirements.txt
├── ContainerOptimizer.spec        ← PyInstaller build spec
├── build_mac.sh
└── build_windows.bat
```

4.2 Module Map

Each module has a single, well-defined responsibility:

Module	Responsibility	Key public interface
`config.py`	Load `optimizer_config.json`; expose constants; fall back to built-in defaults	Module-level constants; `_CONFIG_SOURCE`, `_USING_DEFAULTS`
`parse_xlsx.py`	Read Excel; detect header row; parse pallet rows and NP rows; expand to per-pallet metadata	`parse_pallet_excel_v3()`, `parse_np_boxes_excel_v3()`
`oneDbuildblocks.py`	Snap raw dimensions to canonical footprints; assign height bands; chunk pallets into row-block instances	`build_row_blocks_from_pallets()`, `build_block_type_table()`
`A_1D_multi_container_placement_chatGPT.py`	Define and solve the CP model for one container; expose loaded block indices in order	`RowBlock1DOrderModel` class
`recommend.py`	Greedy 2-D (Y × Z) fill of tail zone and atop zones with pallet blocks and NP boxes	`recommend_fill_containers()`, `print_recommendations()`

`export_excel.py`	Write 4-sheet openpyxl Excel report from solved container data	`export_excel_report()`
`visualize_row_blocks.py`	Render 3-D matplotlib plots of each container's pallet layout + boxes + recommendations	`plot_all_row_block_containers_pallets()`
`main.py`	Orchestrate all steps; handle CLI arguments; write output files	`main()`
`gui.py`	Tkinter desktop front-end; file picker; background run thread; real-time log	`App` class, `run_gui()`

CHAPTER 5

Full Pipeline — Step-by-Step Data Flow

The pipeline is executed in `main.py` in ten sequential steps. Each step receives the output of the previous one.

```
Excel file │ ▼ [STEP 1] parse_xlsx.py │ Detect header row → parse pallet rows → expand to meta_per_pallet (one dict per physical pallet) │ Parse NP rows → np_boxes list (one dict per box type) ▼ [STEP 2] oneDbuildblocks.py │ Snap each pallet to canonical footprint (115×115, 115×108, 115×77, 77×77) │ Assign height band (<66, 66–89, 89–130, >130, 230 cm) │ Chunk into row-block instances (groups of k pallets that form one full row) │ Check all counts are exact multiples → if not, stop and report required additions ▼ [STEP 3–6] CONTAINER SOLVE LOOP (repeats until all blocks are placed) │ │ [STEP 3] Build arrays (lengths, heights, weights, values) from remaining blocks │ [STEP 4] RowBlock1DOrderModel (CP-SAT via CPMpy) │ → which blocks go in, in what order (back→door) │ [STEP 5] Extract solution: block sequence, y-coordinates, leftover length │ [STEP 6] Remove used blocks from remaining list → loop for next container ▼ [STEP 7] assign_boxes_to_containers() (inside main.py) │ Priority 1: fill headroom above each pallet row (atop zones) │ Priority 2: fill tail zone after last pallet row │ Greedy 6-orientation fit; mutates containers in-place ▼ [STEP 8] recommend.py │ For each container: greedy 2-D fill of tail and remaining atop zones │ with row-blocks and NP boxes from same order ▼ [STEP 9] Write outputs │ containers.json, recommendations.json, summary.txt, run.log ▼ [STEP 10] export_excel.py │ Write report.xlsx (Overview / Container Details / Packing Layout / Recommendations) ▼ [STEP 11] visualize_row_blocks.py (optional, disabled in GUI/frozen mode) 3-D matplotlib plots per container
```

Data structures that flow between steps

Object	Type	Description
`meta_per_pallet`	`List[Dict]`	One dict per physical pallet: pallet_id, label, L/W/H (cm), weight_kg, price_fob
`np_boxes`	`List[Dict]`	One dict per NP box type: label, L/W/H (cm), quantity, weight_kg
`blocks`	`List[BlockInstance]`	Row-block instances ready for the solver: block_id, key, length_cm, height_cm, weight_kg, value (pallet count), pallets list
`containers`	`List[Dict]`	Solved container results: rows list, used_length_cm, leftover_cm, loaded_value, loaded_weight, box_zones
`recs`	`List[Dict]`	Recommendation per container: tail_placements, atop_placements, total_pallets_to_add, leftover_before/after_cm

CHAPTER 6

Core Algorithms in Detail

6.1 Excel Parsing (`parse_xlsx.py`)

Header detection

The function `_detect_header_row()` reads the Excel file without a header, then scans each row for overlap with a set of eight known keyword markers. The first row containing at least two of these keywords is used as the header. This makes the parser robust to leading title rows or blank rows above the data table.

Column resolution

Column names are normalised (lowercased, whitespace collapsed) and matched by prefix. For each logical field, an ordered list of candidate names is tried in priority order. This handles spelling variations (e.g. *Productname* vs *Product name*) and column renames across different order export formats.

Dimension parsing

Dimension strings are parsed by `_parse_pallet_size_str()`: strip units, normalise decimal separators, split on `x`, `X`, or Unicode `×`, convert each part from metres to centimetres (multiply by 100, round to integer). The result is always an integer triple `(L_cm, W_cm, H_cm)`.

NP row identification

Rows are flagged as NP loose-box rows when the `Pallet size` type-code column equals exactly `NP` (after stripping and uppercasing). These rows are excluded from the pallet parsing path and processed separately by `parse_np_boxes_excel_v3()`.

6.2 Block Type Classification (`oneDbuildblocks.py`)

Canonical footprint mapping

Each pallet's raw (L, W) dimensions are snapped to the nearest of the three recognised side lengths: **115 cm, 108 cm, 77 cm**, with a tolerance of ±2 cm. If neither dimension maps to a recognised value, the pallet is rejected with a warning. The canonical footprint key is formed as `max(L,W) × min(L,W)` to ensure orientation-independence.

Height band classification

The pallet height is classified into one of five bands, which determine how many pallets can be stacked and therefore how many pallets constitute one row-block:

Band	Height range	Stacking rule
`<66`	h < 66 cm	4 layers high
`66–89`	66 ≤ h ≤ 89 cm	3 layers high
`89–130`	89 < h ≤ 130 cm	2 layers high
`>130`	h > 130 cm (and ≠ 230)	1 layer
`230`	h = 230 cm exactly	1 layer

Block type table

The combination of canonical footprint and height band maps to a pre-defined `BlockType` entry that specifies: pallets per block (k), stack count, conservative block height, and allowed row lengths (rotation variants). The complete table is:

Block key	Pallets/block (k)	Stacks	Block height (cm)	Row length options
`115x115|<66`	8	4	4 × tallest pallet	115 cm
`115x115|66-89`	6	3	3 × tallest pallet	115 cm
`115x115|89-130`	4	2	2 × tallest pallet	115 cm
`115x115|230`	2	1	230 cm	115 cm
`115x108|<66`	8	4	4 × tallest pallet	115 cm or 108 cm
`115x108|66-89`	6	3	3 × tallest pallet	115 cm or 108 cm
`115x108|89-130`	4	2	2 × tallest pallet	115 cm or 108 cm
`115x77|66-89`	6	3	3 × tallest pallet	115 cm or 77 cm
`115x77|89-130`	4	2	2 × tallest pallet	115 cm or 77 cm
`115x77|>130`	2	1	230 cm	115 cm or 77 cm
`77x77|<89`	9	3	3 × tallest pallet	77 cm
`77x77|89-130`	6	2	2 × tallest pallet	77 cm

Block height is computed at instance level as *stack_count × max(pallet heights in chunk)*, giving a conservative (worst-case) height for the container door constraint.

Multiple requirement

Before any block instances are created, the system checks that the pallet count for each block type is an exact multiple of k. If any type has a remainder, the pipeline halts and reports exactly how many additional pallets must be ordered per type to reach the next valid multiple. This is a hard requirement of the row-block model.

Rotation variants

For block types with non-square footprints (e.g. 115×108, 115×77), the algorithm creates two `BlockInstance` objects for each chunk — one with `length_cm = 115` and one with `length_cm = 77` (or 108). Both share the same `block_id`. The solver selects at most one variant per physical block via the no-duplicate constraint. After solving, the unused rotation variant is discarded.

6.3 Constraint Programming Model (`A_1D_multi_container_placement_chatGPT.py`)

The solver uses **CPMpy** (a Python modelling library) over the **Google OR-Tools CP-SAT** backend. The model is a *1-D order-based placement model* for a single container, called repeatedly in an outer loop until all blocks are placed.

Variables

Variable	Domain	Meaning
`slot[r]`	0 … N (integer)	Block instance index at row position r. 0 = empty slot.
`used[r]`	{0, 1} (bool)	True iff slot[r] ≠ 0
`rowsUsed`	0 … R (integer)	Count of occupied row slots
`gapCount`	0 … R (integer)	Number of fork-lift gaps = max(0, rowsUsed−1)
`usedLen`	0 … L (integer)	Total occupied length including gaps

`loadedWeight`	0 … Wmax (integer)	Sum weight of loaded blocks
`loadedValue`	0 … ΣV (integer)	Sum pallet count of loaded blocks
`load[i]`	{0, 1} (bool)	True iff block i appears in any slot (for post-solve extraction)

The number of row slots R (`Rmax`) is automatically computed as the tightest feasible upper bound: ⌊(L + gap) / (min_length + gap)⌋, capped at N (total number of blocks). A 0-indexed sentinel array is prepended to all block property arrays so that `slot[r] = 0` contributes 0 to all sums without branching.

Constraints

Ten constraint groups are enforced:

ID	Constraint	Purpose
C0	`used[r] ↔ (slot[r] ≠ 0)`	Boolean equivalence for used flag
C1	`slot[r] = 0 → slot[r+1] = 0`	Contiguity: all used rows form a prefix (back → door)
C2	`slot[r] ≠ 0 ∧ slot[s] ≠ 0 → slot[r] ≠ slot[s]` for all r < s	Each block placed at most once
C3	`rowsUsed = Σ used[r]`	Count of occupied rows
C4	`gapCount = max(0, rowsUsed − 1)`	One fork-lift gap between each consecutive pair of rows
C5	`usedLen = Σ len(slot[r]) + gap × gapCount ≤ L`	Total length must fit in container
C6	`loadedWeight = Σ weight(slot[r]) ≤ Wmax`	Container maximum payload weight
C7	`loadedValue = Σ value(slot[r])`	Total pallet count (maximised in objective)
C8	`height(slot[r]) ≥ height(slot[r+1])` for all r	Non-increasing heights back → door (stability & visual)
C9	Last used row: `height(slot[r]) ≤ Hdoor`	Tallest block at the door end must clear the door opening
C10	`load[i] ↔ ∃r: slot[r] = i`	Loads flag for post-solve extraction

Objective

The solver maximises a lexicographic combination encoded as a single integer:

```
maximise  10⁶ × loadedValue  +  usedLen
```

The factor of 10^6 ensures that adding even one more pallet always dominates any increase in length utilisation. The length term acts as a tiebreaker that favours placing longer blocks. This means the solver always prioritises loading more pallets over leaving less empty space.

Multi-container loop

After each container is solved, all block instances that share a `block_id` with any loaded block are removed from the remaining list. The model is then rebuilt from scratch for the next container. This continues until the remaining list is empty.

> **Time limit:** The solver is given a wall-clock time limit per container (default 10 seconds, configurable). OR-Tools CP-SAT returns the best feasible solution found within that time. For typical orders (10–50 block types per container), the optimum is usually found in under 2 seconds.

6.4 NP Loose-Box Assignment (`main.py` — `assign_boxes_to_containers()`)

After all containers are solved, a greedy algorithm assigns the NP boxes into the free spaces. A shared pool tracks remaining quantities.

Priority 1 — Headroom zones (atop): For each pallet row in each container, the available headroom is *Hdoor – row_height*. The zone spans the full container width and the row's length along the container. The greedy fill tries all six axis-aligned orientations of each box type and selects the orientation that maximises the number of boxes fitting in that zone (geometric fit). Volume is tracked to limit subsequent box types.

Priority 2 — Tail zone: After processing all rows, the remaining tail length (L – used_length) is filled with the same greedy algorithm using full container height as the zone height.

6.5 Fill Recommendation Engine (`recommend.py`)

The recommendation engine asks: *"If we added more units from the same order, which items would best fill the remaining free space in each container?"*

Candidate generation

Only block types that already appear in the solved container are considered as pallet recommendations (ensuring supply chain alignment). NP box types from the original order are also considered for atop and tail zones.

Scoring

Candidates are scored and sorted by the configured primary objective:

Objective	Score formula	Intent
`min_leftover`	length_cm	Longest block fills the most linear space (default)
`min_pallets`	length_cm / pallets_per_block	Maximum space per pallet ordered
`max_weight`	length × width × height (volume proxy)	Heaviest or densest blocks first
`max_value`	avg_FOB_per_pallet × pallets_per_block	Highest commercial value first

Greedy 2-D fill

The tail zone is a 1-D strip of remaining container length. Candidates are sorted by score, then placed greedily: the highest-scoring block that fits in the remaining length is placed, and the remaining length is decremented (including one fork-lift gap per new block). This repeats until nothing fits.

Atop zones are treated similarly but use a 2-D strategy layered in the height axis: each layer (of height equal to one box height) is filled 1-D across the zone length. No fork-lift gap is applied between boxes in atop zones (they are loose boxes, not fork-lifted pallet rows).

Overlap prevention

Before recommending additions to atop zones, the engine checks which row positions already have NP box zones placed by Step 7 (`atop_covered_y` set). It also checks whether a tail NP zone already exists (`tail_has_np`). Zones already occupied are skipped to prevent double-counting.

CHAPTER 7

Configuration Reference

All tunable parameters are stored in `optimizer_config.json` placed next to the executable (or project root in development). If the file is absent, built-in defaults are used and a warning is printed at every run. The file is plain JSON and can be edited with any text editor.

```
{
  "CONTAINER_LENGTH_CM":           1203,
  "CONTAINER_WIDTH_CM":            235,
  "CONTAINER_HEIGHT_CM":           270,
  "CONTAINER_DOOR_HEIGHT_CM":      250,
  "CONTAINER_MAX_WEIGHT_KG":       18000,
  "ROW_GAP_CM":                    5,
  "SOLVER_TIME_LIMIT_SEC":         10,
  "RECOMMEND_OBJECTIVE":           "min_leftover",
  "RECOMMEND_SECONDARY_OBJECTIVE": "min_pallets"
}
```

Parameter	Default	Description
`CONTAINER_LENGTH_CM`	1203	Internal usable length of the container (back wall to door). Standard 40-ft HC = 1203 cm.
`CONTAINER_WIDTH_CM`	235	Internal usable width. Standard 40-ft HC = 235 cm.
`CONTAINER_HEIGHT_CM`	270	Internal usable height. Used for 3-D plots only; does not constrain packing.
`CONTAINER_DOOR_HEIGHT_CM`	250	Door opening height. The last row at the door end must be ≤ this height. Standard = 250 cm.
`CONTAINER_MAX_WEIGHT_KG`	18000	Maximum payload weight per container. If no pallet weights are in the Excel, this constraint is never binding.
`ROW_GAP_CM`	5	Fork-lift clearance gap between consecutive pallet row-blocks (cm). Added once between each pair of adjacent rows.
`SOLVER_TIME_LIMIT_SEC`	10	Maximum wall-clock time in seconds given to the CP-SAT solver per container. Increase for very large orders or complex block mixes.
`RECOMMEND_OBJECTIVE`	`min_leftover`	Primary objective for the recommendation engine. Options: `min_leftover`, `min_pallets`, `max_weight`, `max_value`.
`RECOMMEND_SECONDARY_OBJECTIVE`	`min_pallets`	Tiebreaker objective when two candidates have equal primary scores.

Hardcoded parameters (require code changes):
Recognised pallet footprint side lengths: 115, 108, 77 cm (±2 cm tolerance).
Height band boundaries: 66, 89, 130, 230 cm.
Stacking rules per band (pallets per block, stack count) — see §6.2 table.
These are domain-specific constants that reflect the physical pallet standards in use. They can be changed in `oneDbuildblocks.py → build_block_type_table()`.

CHAPTER 8

Output Files

All output files are written to an `outputs/` subfolder next to the Excel input file (or next to the executable when running frozen).

8.1 report.xlsx

The main deliverable. Four worksheets:

Sheet	Contents
Overview	Key metrics (containers used, total pallets, NP boxes, weight, average volumetric fill). Per-container summary table with traffic-light volumetric fill % (green ≥85%, amber ≥60%, red <60%). Bar chart of fill % per container.
Container Details	Full row-by-row breakdown for every container: each pallet row-block (type, length, height, weight, pallet count, position) and each NP box zone (zone type, box label, dimensions, quantity).
Packing Layout	Colour-coded cell grid. Each spreadsheet column ≈ 12 cm of container length. Rows represent layers: pallets on top, NP box zones, then recommendation additions (+). Colours identify footprint type.
Recommendations	Per-container list of items to add (pallet blocks and NP boxes) showing zone (TAIL / ATOP), length, height, count, total units, and estimated FOB value where available.

Volumetric fill %

The fill percentage shown throughout the report is **volumetric**, not length-based. It is computed as:

```
vol_fill % = ( Σ[row.length × W × row.height]  +  Σ[NP box zone volume] )
             ──────────────────────────────────────────────────────────────  × 100
                        Container_L × Container_W × Door_H
```

This correctly penalises containers loaded with a single thin layer of short pallets, unlike a length-only metric that would show such a container as nearly full.

8.2 JSON and Log Files

File	Description
`containers.json`	Complete solved container data in machine-readable form. Contains every row-block position, all box zones, weights, and pallet metadata. Useful for integration with other systems.
`recommendations.json`	Recommendation results: per-container tail and atop placements, aggregated counts, estimated FOB values.
`summary.txt`	Human-readable one-page summary: number of containers, total pallets, total weight, per-container used length and leftover.
`run.log`	Timestamped log of every step, assumption, and warning printed during the run. Useful for auditing and debugging.

CHAPTER 9

Known Assumptions & Limitations

Every assumption the system makes is printed to the log at the start of each run. The most important ones are summarised here.

Geometric assumptions

- **Pallets occupy the full container width.** A row-block is modelled as a rectangular slab of width = container width (235 cm). The model does not consider partial-width rows or pallets placed lengthwise.
- **All pallets within a row are identical in footprint and height.** The row-block model requires pallets to be grouped by type. Mixed-type rows are not supported.
- **Container dimensions are interior clearances.** Any structural members, lashing rails, or wall thickness must be accounted for when setting the config values.
- **Pallet heights are the loaded height** (pallet + stacked goods). The reported block height uses the tallest pallet in each chunk as the conservative bound.

Solver assumptions

- **Sequential container packing.** Containers are solved one at a time in order. The optimal global solution (all containers simultaneously) is not computed. For most order sizes this gives results very close to optimal.
- **Height ordering back → door is enforced.** The tallest rows go to the back of the container, the shortest (under door height) are at the door. This is a physical loading constraint.
- **Time-limited solve.** With the default 10-second limit, the solver may not reach a proven optimum for very large instances. The best feasible solution found within the time limit is used.

Recommendation limitations

- Recommendations only suggest items already in the current order. No new product types are introduced.
- The greedy algorithm is not globally optimal — it picks the locally best fit at each step.
- NP box recommendations do not account for box fragility or stacking weight limits.

Input limitations

- All pallet counts must be exact multiples of the pallets-per-block value for their type. The system will halt and report required additions if this condition is not met.
- Only the footprint sizes 115, 108, and 77 cm (±2 cm) are recognised. Pallets outside this range are silently skipped.
- Weight data is optional but if absent the weight constraint is never binding, which may lead to overloaded containers in edge cases.

APPENDIX A

User Manual

> **You do not need to understand any of the technical content in this document to use the software.** This appendix is the only section you need to read.

What does this tool do?

You give it your order Excel file. It tells you how many shipping containers you need, exactly how to pack them, and what extra items you can add to fill the remaining space. It also generates a colour-coded Excel report you can share with your logistics team.

Step 1 — Prepare your Excel file

The tool reads your standard order export. Make sure it has these columns (the exact names do not need to match perfectly — the tool tries several common variations):

- **Pallet and packing size** — the physical dimensions of each pallet, e.g. `1.15x1.15x1.01` (length × width × height in metres).
- **External Packaging Quantity** (or *Total order full pallets*) — how many full pallets of that product you have ordered.

Optionally, also include:

- **Productname** — shown in the report as human-readable labels.
- **External Net weight** — weight per pallet in kg, used to check the container does not exceed its weight limit.
- For loose boxes (not palletised), mark the row with `NP` in the **Pallet size** column and put the box dimensions in **Pallet and packing size**.

> **Important:** The total number of pallets for each product must be a multiple of a specific number (depending on pallet size — usually 2, 4, 6, 8, or 9). If it is not, the tool will stop and tell you exactly how many more pallets to add. This is a physical requirement of the packing method, not a software limitation.

Step 2 — Open the application

Windows: Double-click `ContainerOptimizer.exe` in the application folder.
Mac: Double-click `ContainerOptimizer.app`.
From Python: Run `python app/gui.py` in a terminal.

The application window opens. It looks like this:

```
┌──────────────────────────────────────────────────────────┐
│ Container Packing Optimizer                   Config: ...  │
├──────────────────────────────────────────────────────────┤
│ Excel Input File                          [Browse...]     │
│ No file selected                                          │
├──────────────────────────────────────────────────────────┤
│ [Run Optimizer]  ▓▓▓▓▓▓▓▓▒▒▒▒  Status: Ready              │
├──────────────────────────────────────────────────────────┤
│ Run Log                                                   │
│ ┌──────────────────────────────────────────────────────┐ │
│ │                                                      │ │
│ └──────────────────────────────────────────────────────┘ │
├──────────────────────────────────────────────────────────┤
│ [Open Report (Excel)]  [Open Output Folder]               │
└──────────────────────────────────────────────────────────┘
```

Step 3 — Select your file

Click **Browse...**. A file picker opens. Navigate to your order Excel file and click Open. The file path will appear under the button.

Step 4 — Run the optimizer

Click **Run Optimizer**. The progress bar starts moving and the Run Log area fills with live output. Depending on the size of your order, this takes between 10 seconds and a few minutes. Do not close the window.

You will see messages like:

```
Parsed 340 physical pallets
Constructed 42 row-block VARIANTS
--- Solving container 1 ---
Loaded blocks: 12  |  Used: 1198 / 1203 cm
--- Solving container 2 ---
...
```

Step 5 — Check the result

When the log ends with *"Done! Outputs saved to: ..."*, click **Open Report (Excel)**. Your report opens in Microsoft Excel with four sheets:

Sheet	What to look at
Overview	At a glance: how many containers, total pallets, average fill %, and the bar chart. Green = well-packed, amber = OK, red = significant space wasted.
Container Details	Exactly which pallet types go in which container and where. Share with the warehouse team for loading instructions.
Packing Layout	Visual colour-coded map of each container. Each coloured cell block is one pallet row. Green cells show recommended additions.
Recommendations	If any container has free space, this lists the exact items you should add to the order to fill it. Includes how many units and which zone they go in.

Step 6 — Review recommendations (optional)

The last container is often partially empty. Check the **Recommendations** sheet — if the suggested additions make commercial sense, you can increase those line items in your order and re-run the tool to confirm they fit.

Changing the container size or solver settings

Open `optimizer_config.json` (it is in the same folder as the application) with Notepad or any text editor. Change the values and save. The next run will use the new settings. The top of the Run Log always shows which settings are active.

The most commonly changed settings are:

- `CONTAINER_LENGTH_CM` — if you are using a different container size (e.g. 20-ft = 590 cm).
- `CONTAINER_MAX_WEIGHT_KG` — if your carrier has a lower payload limit.
- `SOLVER_TIME_LIMIT_SEC` — increase to 30 or 60 for very large orders if the result looks suboptimal.

Troubleshooting

Problem	Likely cause & fix

Tool stops with *"ORDER NOT VALID FOR FULL ROW-BLOCK MODEL"*	Pallet counts are not exact multiples. The log will say e.g. *"115x115|89-130: add 2 pallets"*. Add that many pallets to the order for that product and re-run.
No NP boxes appear in the output	Check that the **Pallet size** column (not Pallet and packing size) contains exactly `NP` for those rows, and that the quantity column is filled in.
Container 9 (or last container) is mostly empty	This is normal — the last container holds whatever pallets did not fit in the previous ones. Use the Recommendations sheet to see what to add to fill it.
Report not opening	Click **Open Output Folder** and find `report.xlsx` manually. Open it with Excel. If it does not exist, check the Run Log for any error in red.
Application closes immediately on double-click (Windows)	Windows Defender may have quarantined the executable. Right-click → Properties → Unblock, or add the folder to antivirus exclusions.
Very slow on large orders	Increase `SOLVER_TIME_LIMIT_SEC` is not the issue here — the solver is fast. Slowness usually comes from 3-D plotting if running from Python. Use the GUI (`gui.py`) which disables plots automatically.